WEB #
START

Introduction aux CSS

Vincent Caronnet // 2021-2022
Formateur web & mobile

Ce cours vous est présenté par...

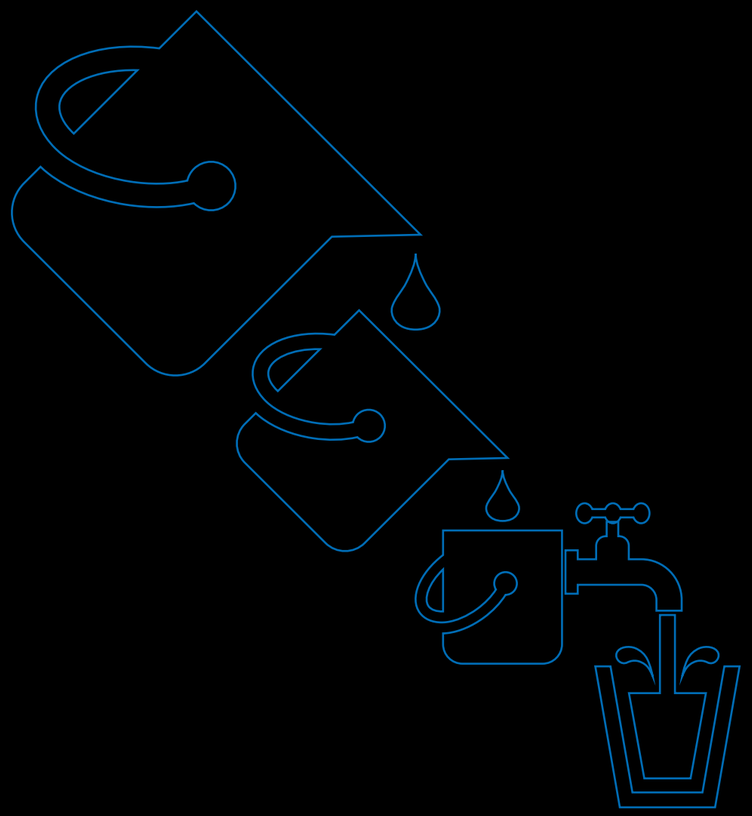
Vincent Caronnet, développeur web et infographiste depuis 1996, j'interviens à **Webstart** et **MJM** depuis 2015 en tant que :

- **Webdesigner** (Figma, Adobe XD, HTML5, CSS3, JS, WordPress)
- **Développeur mobile** (Swift, React Native)
- **Développeur web *full stack*** (Node.js, Express, React)
- **Graphiste print** (InDesign, Illustrator, Photoshop)



120@120.design @120design @apps.120design

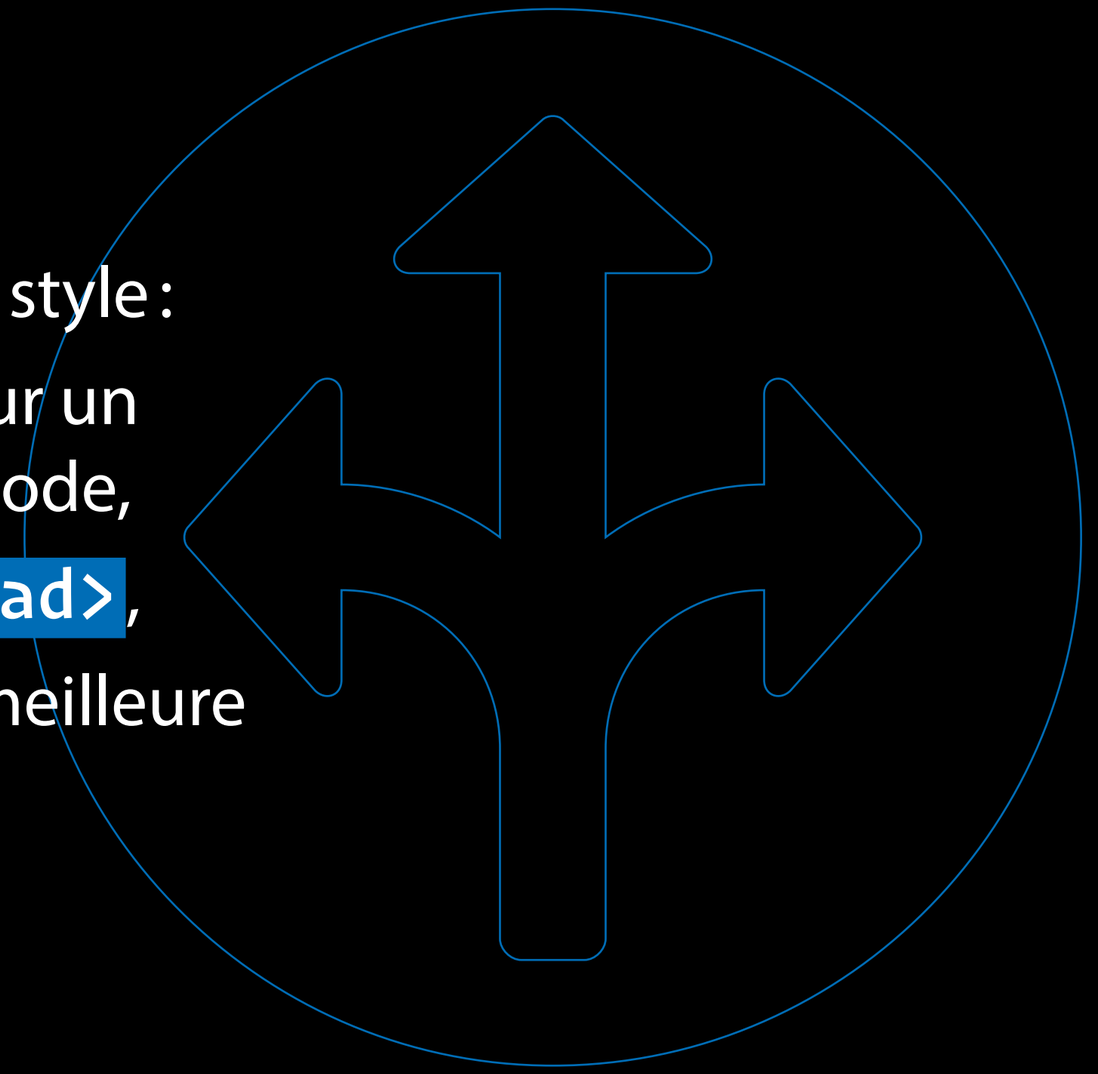
Les CSS, des feuilles de styles en cascade



Les feuilles de styles en cascade (*Cascading Style Sheets*) sont apparues dans la deuxième moitié des années '90 pour **décrire au navigateur la présentation voulue d'un contenu HTML**.

On applique une feuille de style à **un élément HTML**. On dit que les feuilles de styles **s'appliquent "en cascade"** à cet élément parce que plusieurs feuilles de styles **peuvent impacter simultanément** la présentation de cet élément.

Une autoroute à trois voies



Il existe trois méthodes pour charger un style :

- créer **un style en ligne** directement sur un élément HTML, la plus mauvaise méthode,
- créer **un style embarqué** dans le `<head>`,
- lier **une feuille de styles externe**, la meilleure méthode.

Les styles en ligne

```
<body>
  <h1>Mon titre principal</h1>
  <ul>
    <li style="color:red;font-family:sans-serif;">Premier élément de la liste</li>
    <li>Deuxième élément de la liste</li>
    <li>Troisième élément de la liste</li>
  </ul>
</body>
```

Règles CSS



Un style en ligne consiste à ajouter l'attribut `style` à l'élément HTML dont on veut définir la présentation.

L'attribut `style` d'une balise HTML a pour valeur une ou plusieurs **règles CSS** séparées par un `;`.

Les styles en ligne

C'est la plus mauvaise façon de styler un élément :

- le code HTML qui ne devrait servir qu'à structurer le contenu de la page contient des attributs de styles, **le fond** (HTML) **et la forme** (CSS) **sont mélangés**,
- on ne peut pas appliquer un style à plusieurs éléments d'un coup, **on est obligé de se répéter**.
- le code CSS, dont les même styles se répètent et sons noyés dans le HTML, **est difficilement maintenable**.

```
<h1>Mon titre principal</h1>
<ul>
  <li style="color:red;
  font-family:sans-serif;">
    Premier élément de la liste
  </li>
  <li style="color:red;
  font-family:sans-serif;">
    Deuxième élément de la liste
  </li>
  <li style="color:red;
  font-family:sans-serif;">
    Troisième élément de la liste
  </li>
</ul>
```

Les styles embarqués

```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="description"
  content="Bla bla">
  <title>Titre du document</title>
  <style>
    li {
      color:red;
      font-family:sans-serif;
    }
  </style>
</head>
<body>
  <h1>Mon titre principal</h1>
  <ul>
    <li>Premier list item</li>
    <li>Deuxième list item</li>
```

Les styles embarqués, ce sont les des styles qu'on déclare dans une balise `<style>`, dans le `<head>`.

Ces styles s'appliqueront **à toute la page HTML** :

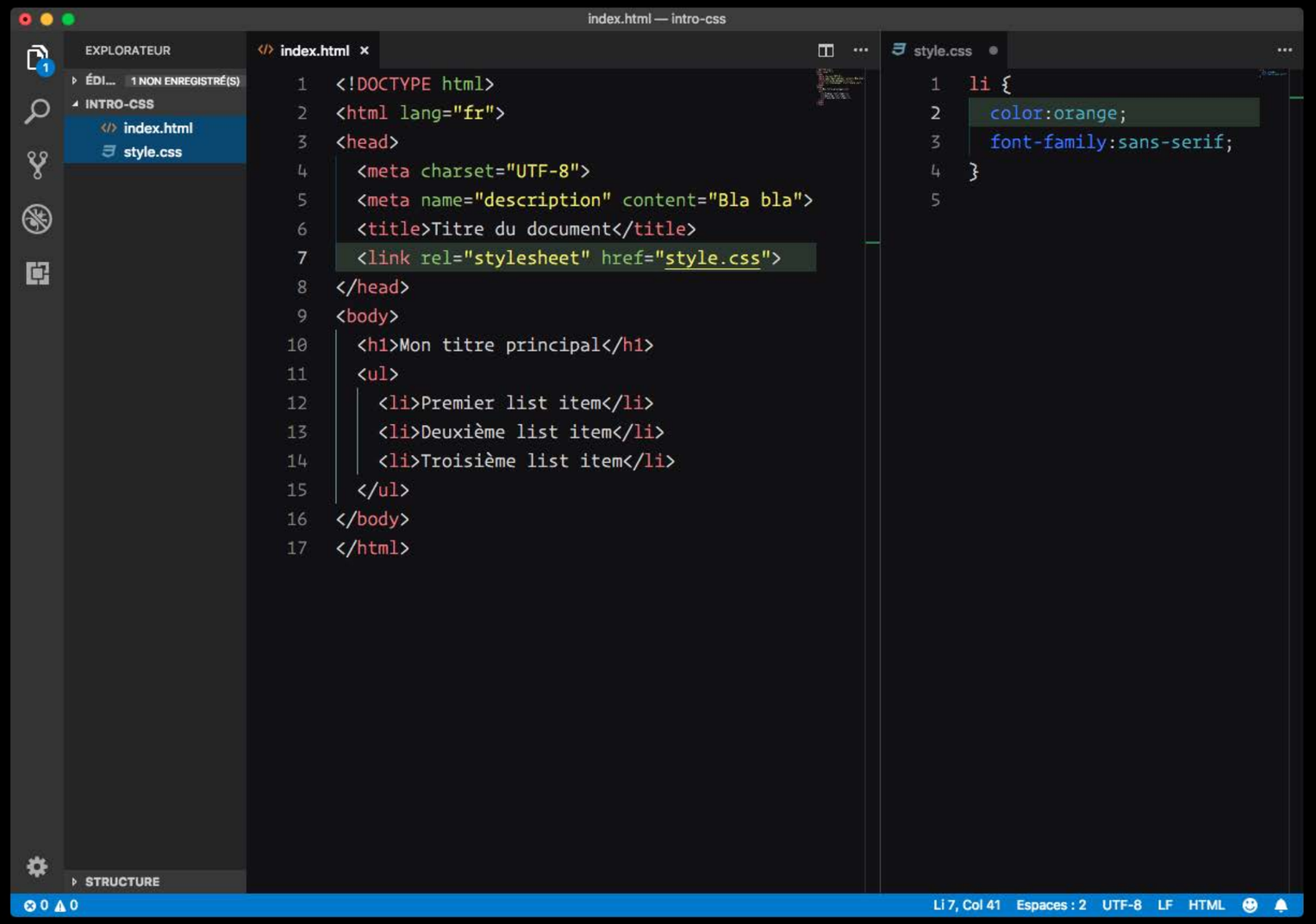
- on n'aura **pas besoin de répéter** les styles,
- les CSS **ne seront pas noyés** dans le HTML,
- mais il sera **compliqué d'appliquer les mêmes styles à plusieurs pages** d'un même site, puisqu'il faudra les copier-coller dans chaque page, puis **répercuter leurs éventuelles modifications** dans toutes les pages.

Introduction aux CSS

Les feuilles de styles liées

La meilleure pratique est de :

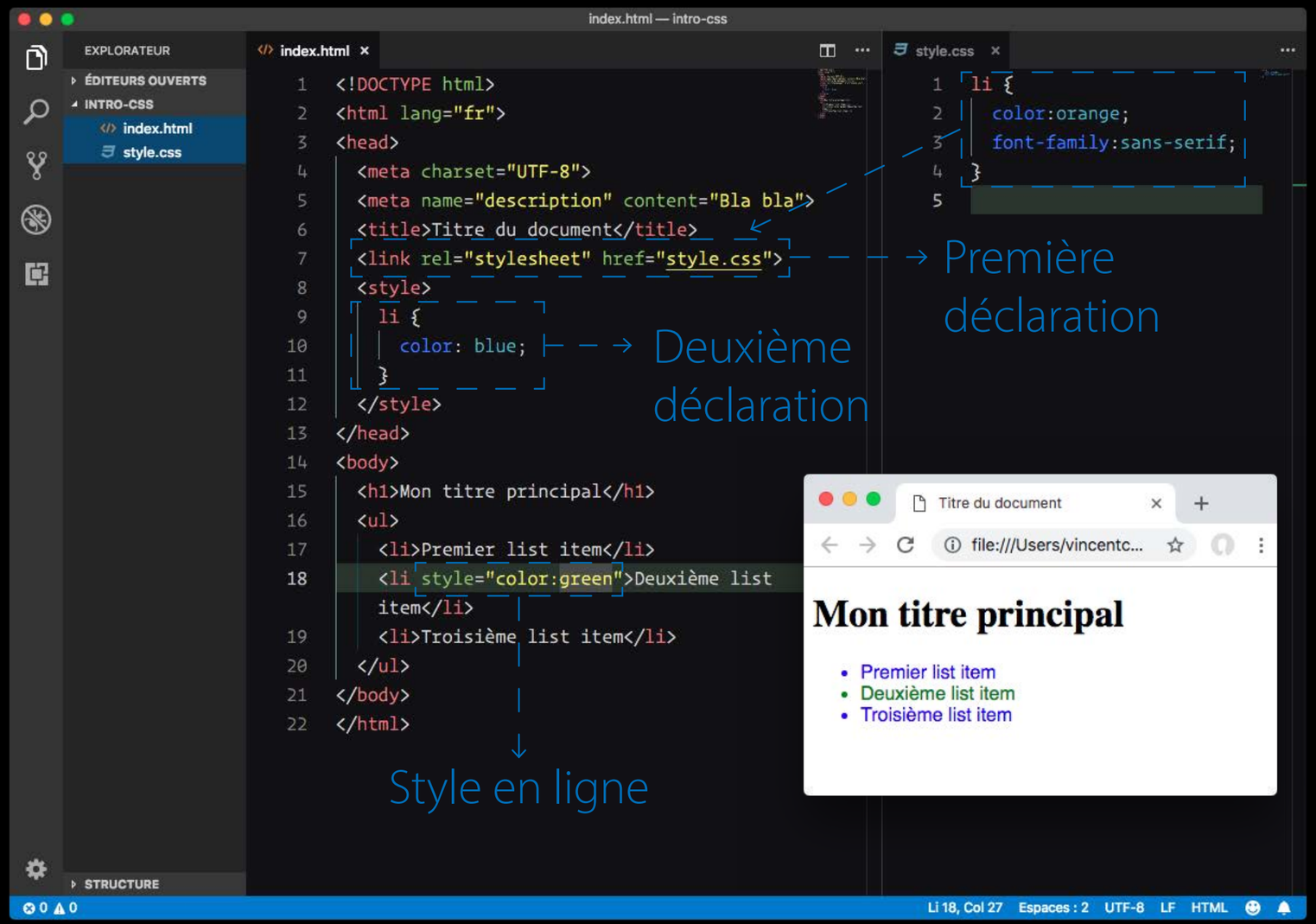
- **créer un fichier .css** dans lequel on écrit des règles CSS,
- **charger ce fichier .css** dans le `<head>` du fichier .html, en utilisant une balise `<link>` et son attribut `href`.



Vincent Caronnet // 2021-2022
Formateur web & mobile

Introduction aux CSS

Priorité des règles CSS



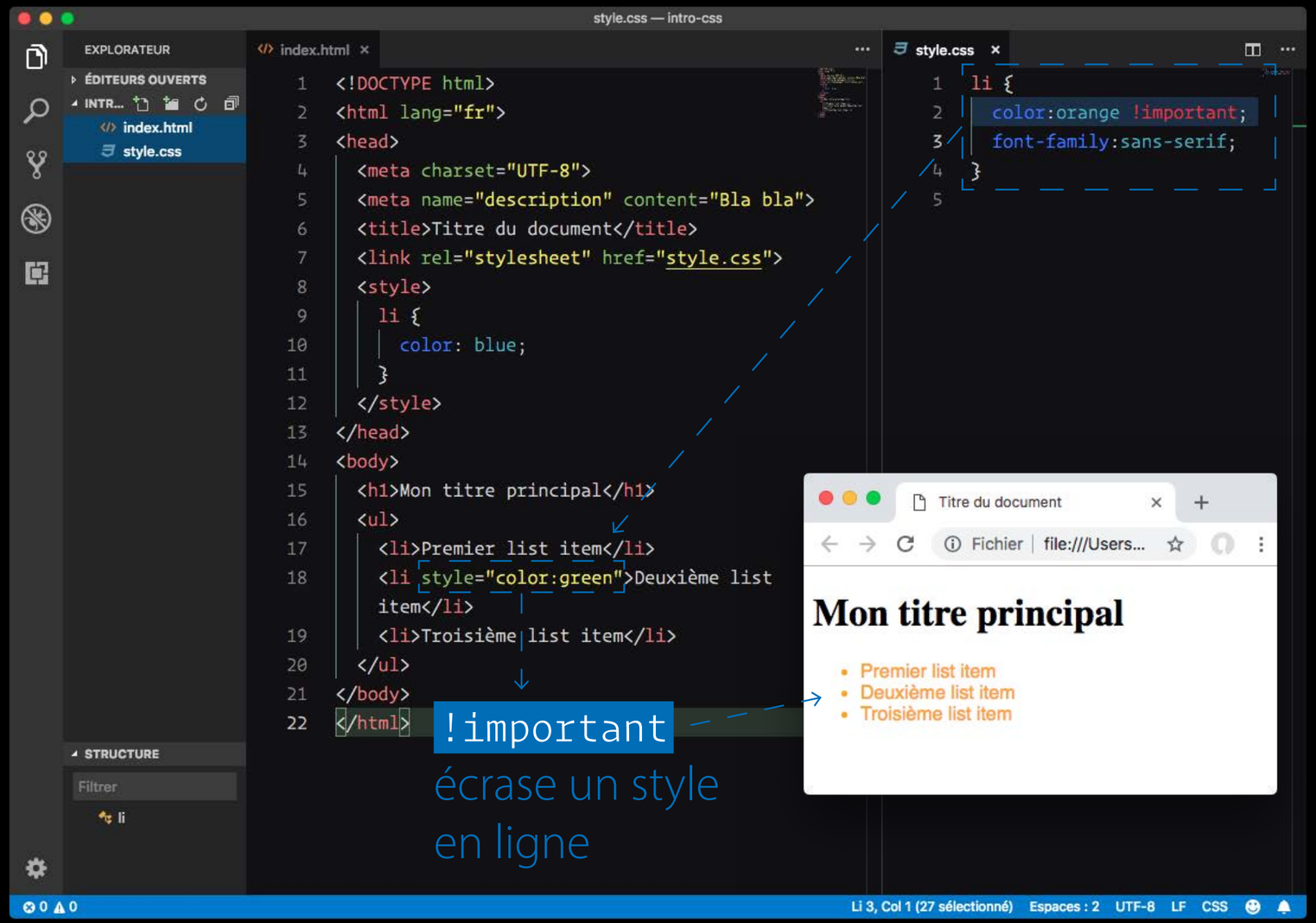
En cas de déclaration de **deux règles identiques** :

- **les styles en ligne priment** sur tout autre style,
- **la dernière déclaration** d'un style lié ou embarqué **écrase les précédentes**.

WEB # START

Introduction aux CSS

Priorité des règles CSS



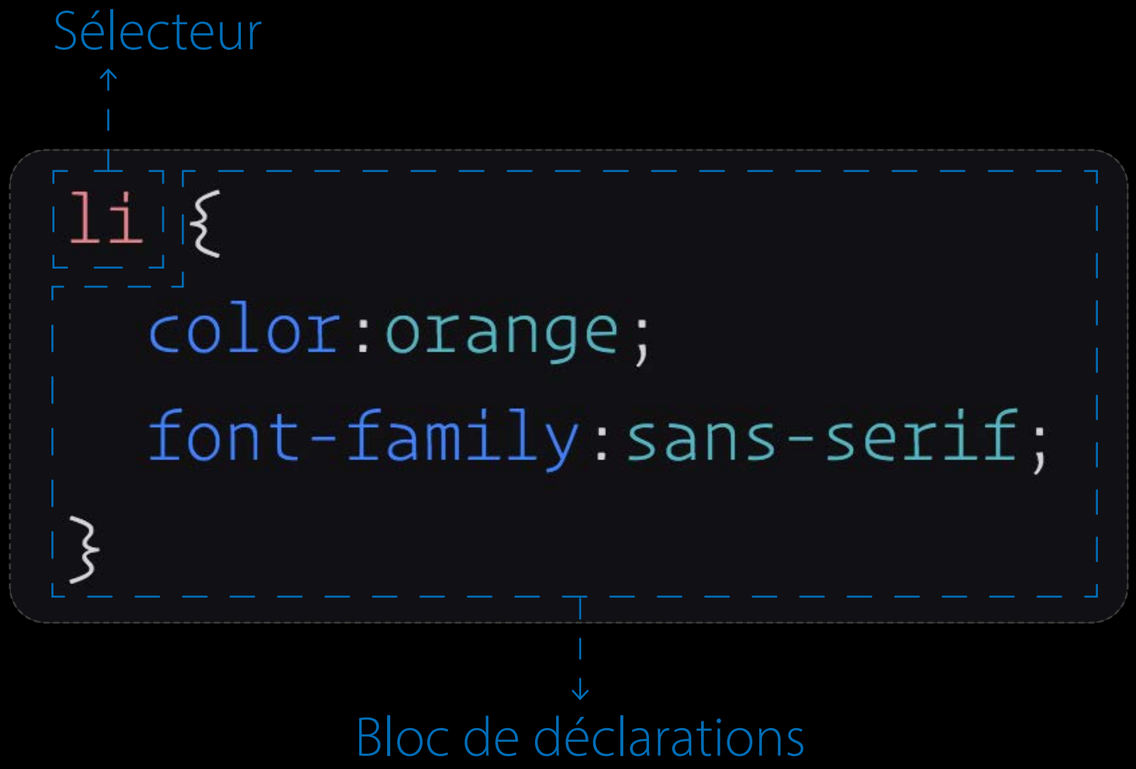
La seule façon d'**écraser toutes les autres déclarations**, y compris les styles en ligne, est d'ajouter `!important` à la fin d'une règle CSS.

Il faut **le faire en dernier recours**, car cela nuit à la maintenabilité du code !

Anatomie d'une règle CSS

Une règle CSS se divise en deux parties :

- **un sélecteur,** avant les accolades, qui cible le ou les éléments concernés par la règle CSS,
- **un bloc de déclarations,** délimité par les accolades.

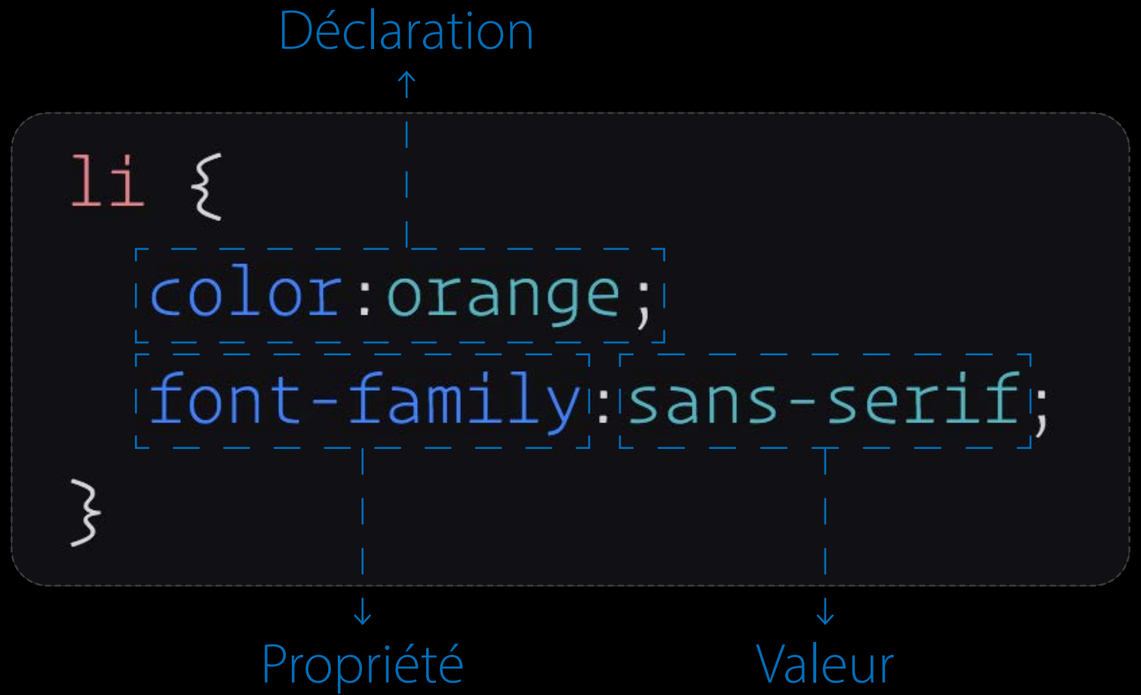


Anatomie d'une règle CSS

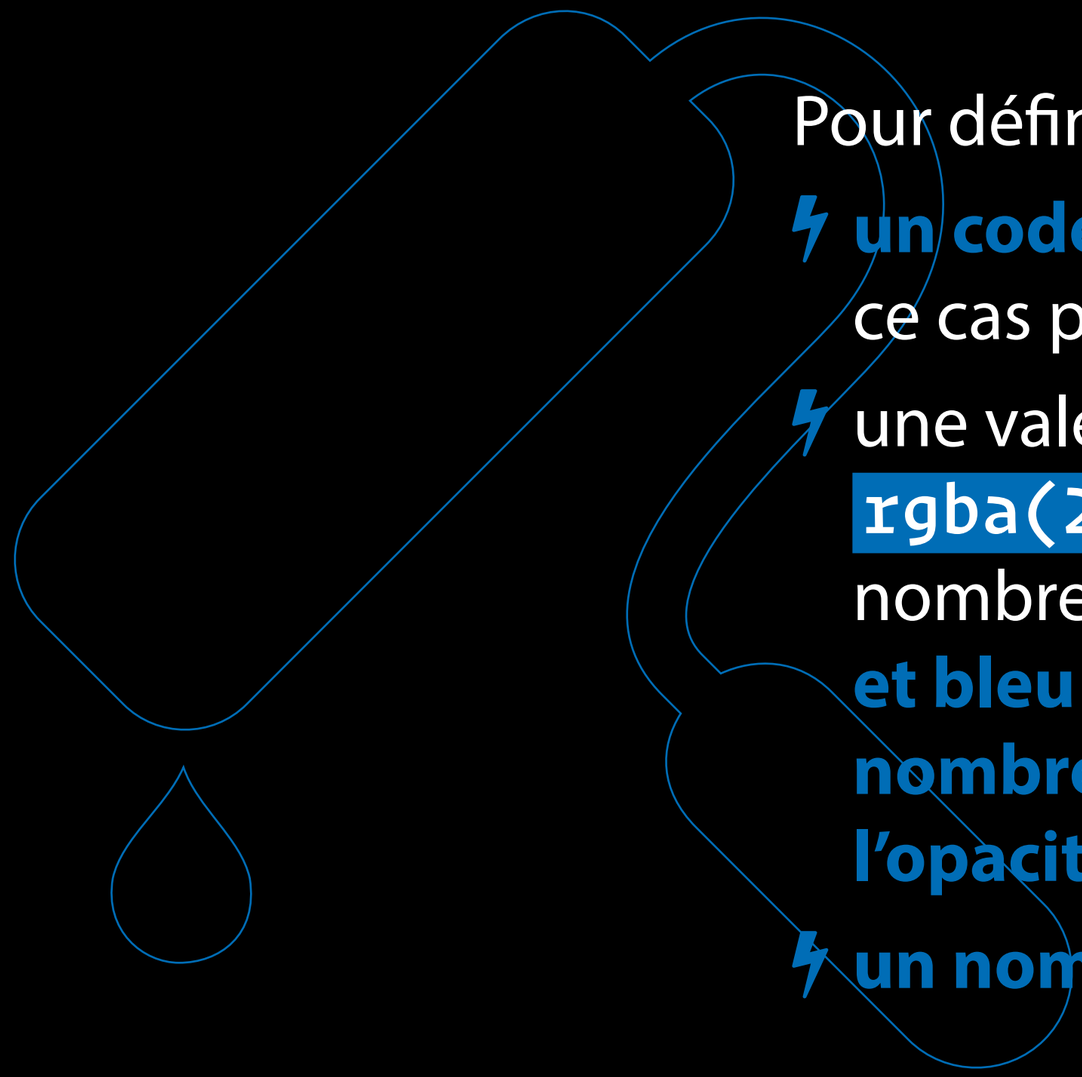
À l'intérieur du bloc de déclarations, chaque déclaration **doit obligatoirement finir par un ;**, sinon les déclarations suivantes seront ignorées par le navigateur !

Chaque déclaration est divisée en deux parties, délimitées par **:** :

- **une propriété**, avant les **:**,
- **une valeur**, après les **:** et avant le **;** final.



Définir la valeur d'une couleur



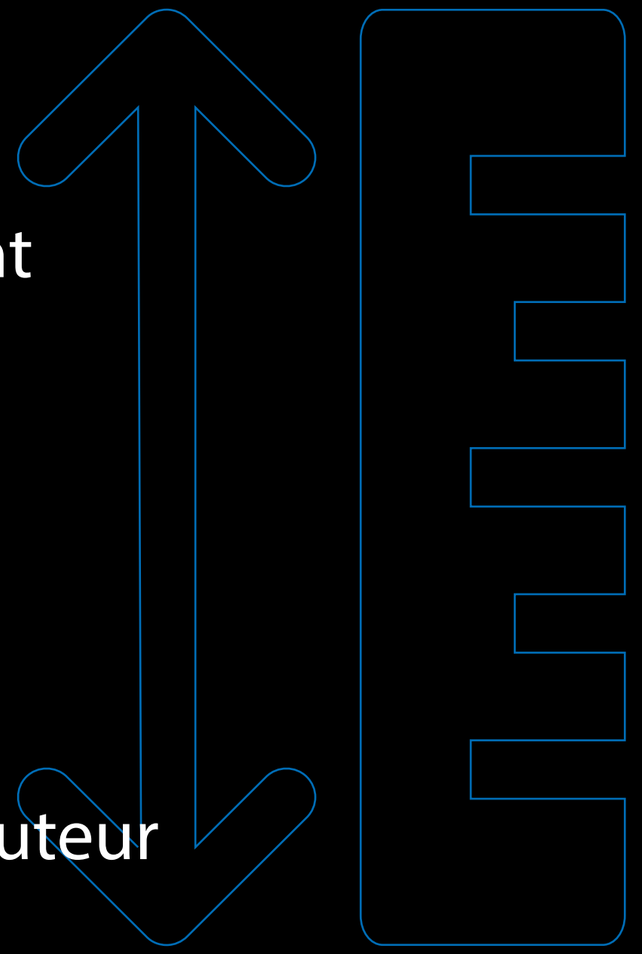
Pour définir une couleur en CSS, on peut utiliser :

- **un code hexadécimal** (ex. : `#ff0000`, qui dans ce cas peut être raccourci en `#f00`),
- une valeur **rgb()** ou **rgba()** (ex. : `rgba(255,0,0,0.8)`) où les trois premiers nombres expriment les valeurs de **rouge, vert et bleu entre 0 et 255**, auxquels on ajoute **un nombre compris entre 0 et 1** pour exprimer **l'opacité** si on utilise le rgba(),
- **un nom de couleur** en anglais (ex. : `yellow`).

Définir la valeur d'une dimension

En CSS, les dimensions s'expriment en :

- `px`, des pixels (ex. : `100px`),
- `%`, un pourcentage se référant au parent de l'élément dimensionné (ex. : `50%`),
- `em`, un coefficient se référant à la taille de police de l'élément dimensionné (ex. : `1.2em`),
- `rem`, un coefficient se référant à la taille de police de la l'élément racine `<html>` (ex. : `1.2rem`),
- `vw` ou `vh`, un pourcentage de la largeur ou de la hauteur du viewport, la fenêtre du navigateur (ex. : `80vh`).



Styliser du texte

Les principales propriétés CSS pour formater du texte sont :

- `color` pour la couleur, exprimée en hexadécimal, en `rgb()`, etc.,
- `text-align` pour la justification,
- `font-family` pour la police de caractères,
- `font-size` pour le corps, exprimée en `px`, `em`, `rem`, etc.,
- `font-weight` pour la graisse, exprimée soit par un mot-clé (`normal`, `bold`, etc.), soit par un nombre compris entre `1` et `1000`, du plus léger au plus gras,
- `line-height` pour l'interlignage, exprimée en `px`, `em`, `rem`, etc.

La propriété text-align

La propriété `text-align` accepte une de ces quatre valeurs :

- `left` pour ferrer le texte à gauche,
- `right` pour ferrer le texte à droite,
- `center` pour centrer le texte,
- `justify` pour justifier le texte, mais cela n'est pas recommand, car cela peut provoquer des lézardes en fonction de la taille de la fenêtre.

La propriété font-family

`font-family`, la propriété CSS qui définit la police de caractères d'un texte, accepte notamment ces mots-clés pour valeur :

- `serif`, la valeur par défaut, demande au navigateur d'afficher une police à empattement disponible sur l'ordinateur de l'utilisateur,
- `sans-serif`, avec lequel le navigateur affichera une police sans empattement,
- `monospace`, avec lequel le navigateur affichera une police à chasse fixe,
- `cursive`, pour afficher une police manuscrite.



La propriété font-family

On peut demander au navigateur d'afficher une police en particulier, en donnant en valeur à la propriété `font-family` le nom de cette police.

Ce nom doit être entre guillemets `""` s'il comporte des espaces.

Comme on ne peut pas savoir si l'utilisateur a cette police installée sur son ordinateur, il est conseillé d'indiquer à la suite de cette police des polices de secours qui seront utilisées en cas d'indisponibilité :

```
font-family: "Open Sans", Arial, sans-serif;
```

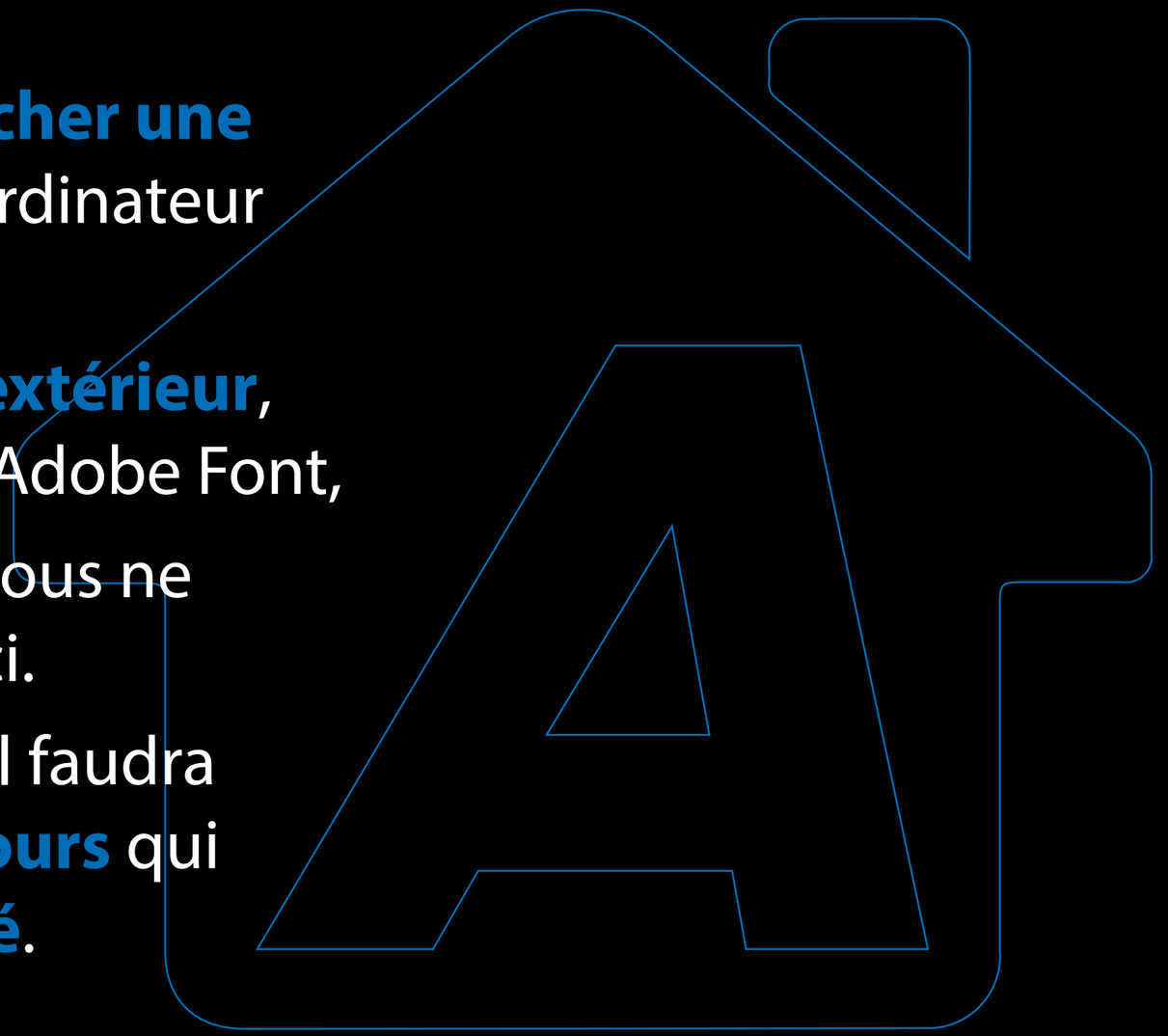


La propriété font-family

Pour demander au navigateur d'**afficher une police qui n'est pas installée** sur l'ordinateur de l'utilisateur, il faut soit :

- charger **une police hébergée à l'extérieur**, par exemple par Google Fonts ou Adobe Font,
- **héberger soi-même** une police, nous ne détaillerons pas cette procédure ici.

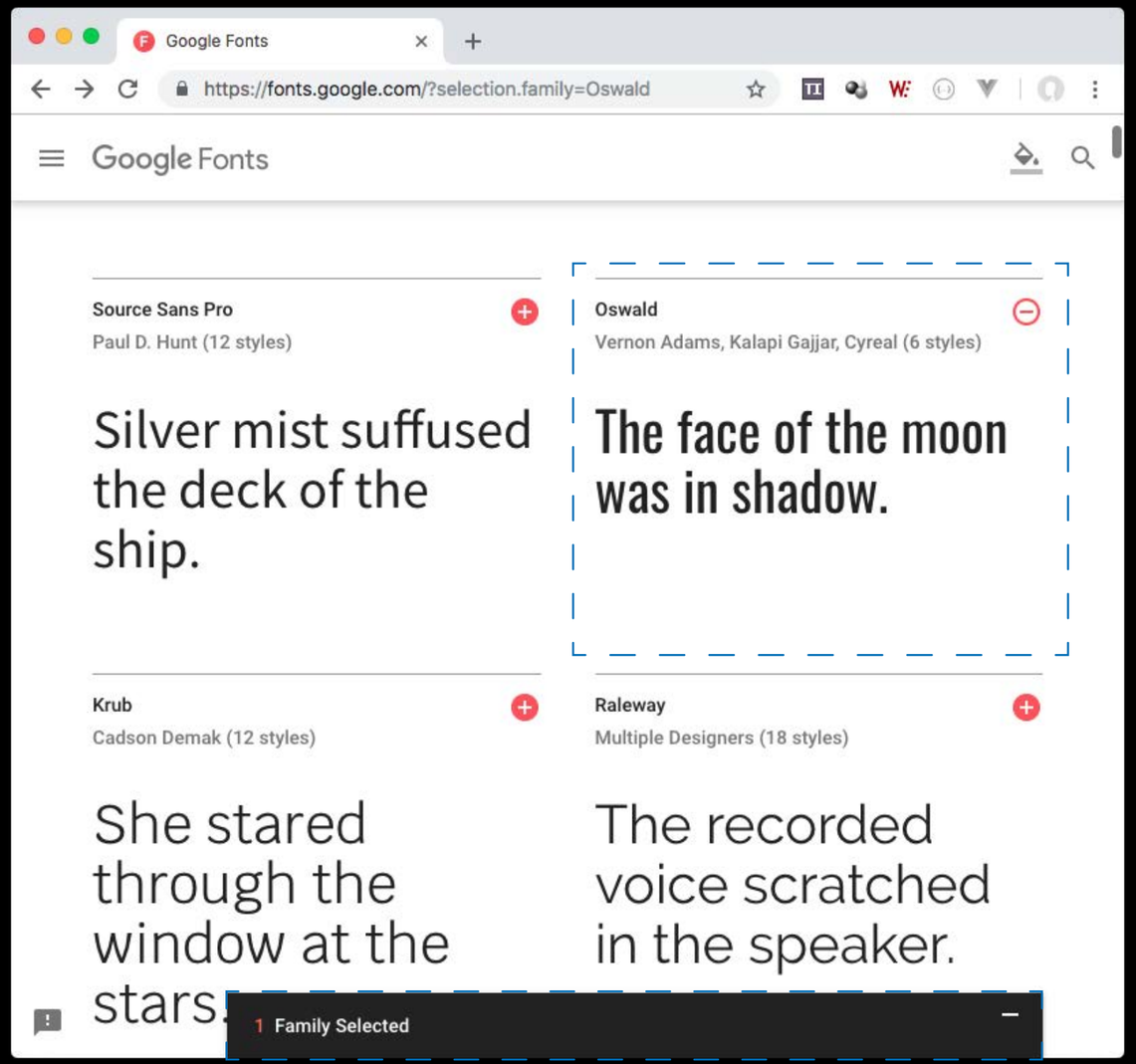
Quelle que soit la méthode choisie, il faudra veiller à indiquer **une police de secours** qui sera utilisée **en cas d'indisponibilité**.



Utiliser une police hébergée par Google

Voici un exemple d'utilisation d'une police Google :

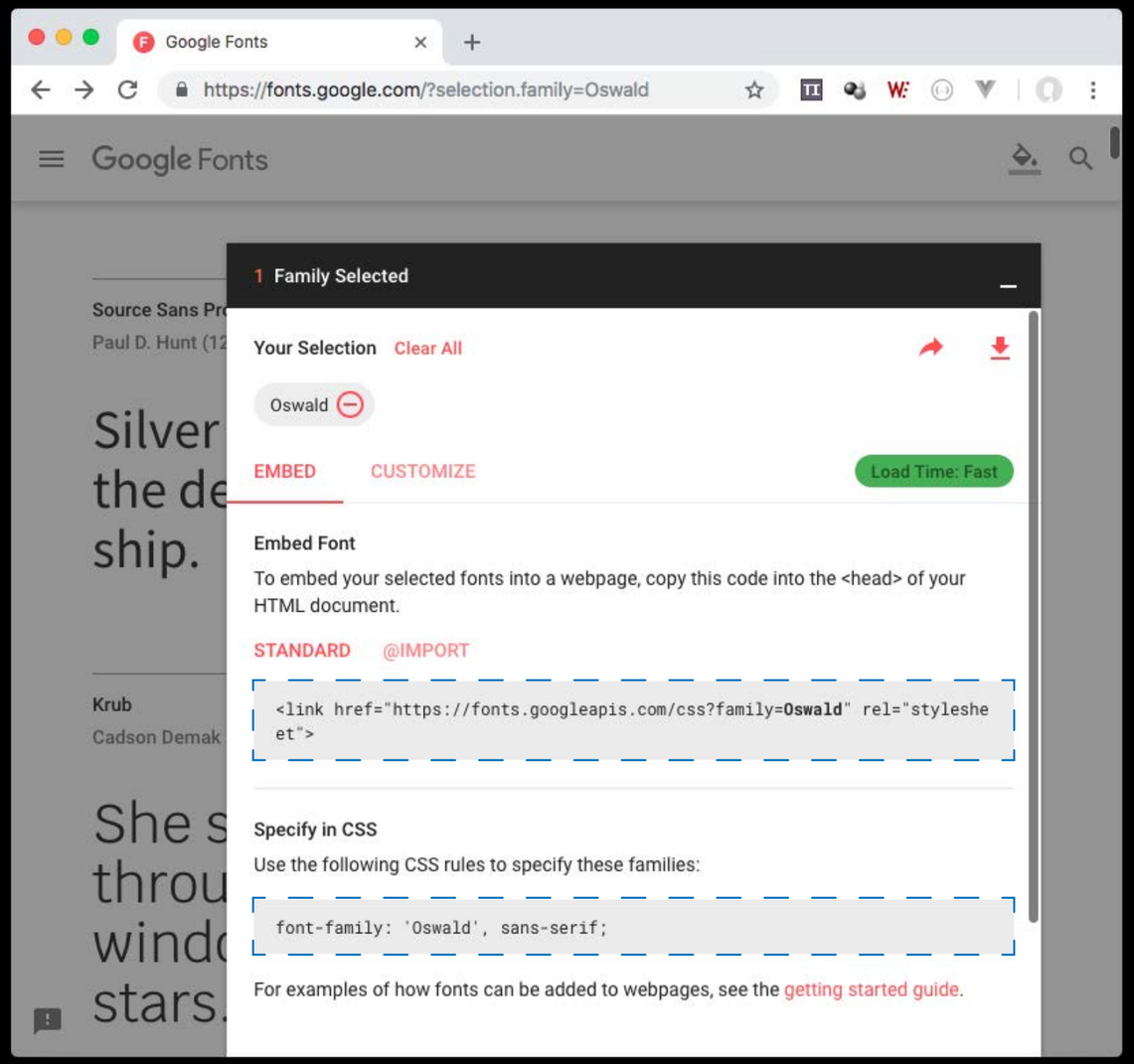
- **nous avons cliqué sur le "+"** de la police Oswald,
- **cette famille est ajoutée** aux famille sélectionnées,
- **l'onglet "Family Selected"** est apparu **en bas de la fenêtre** du navigateur.



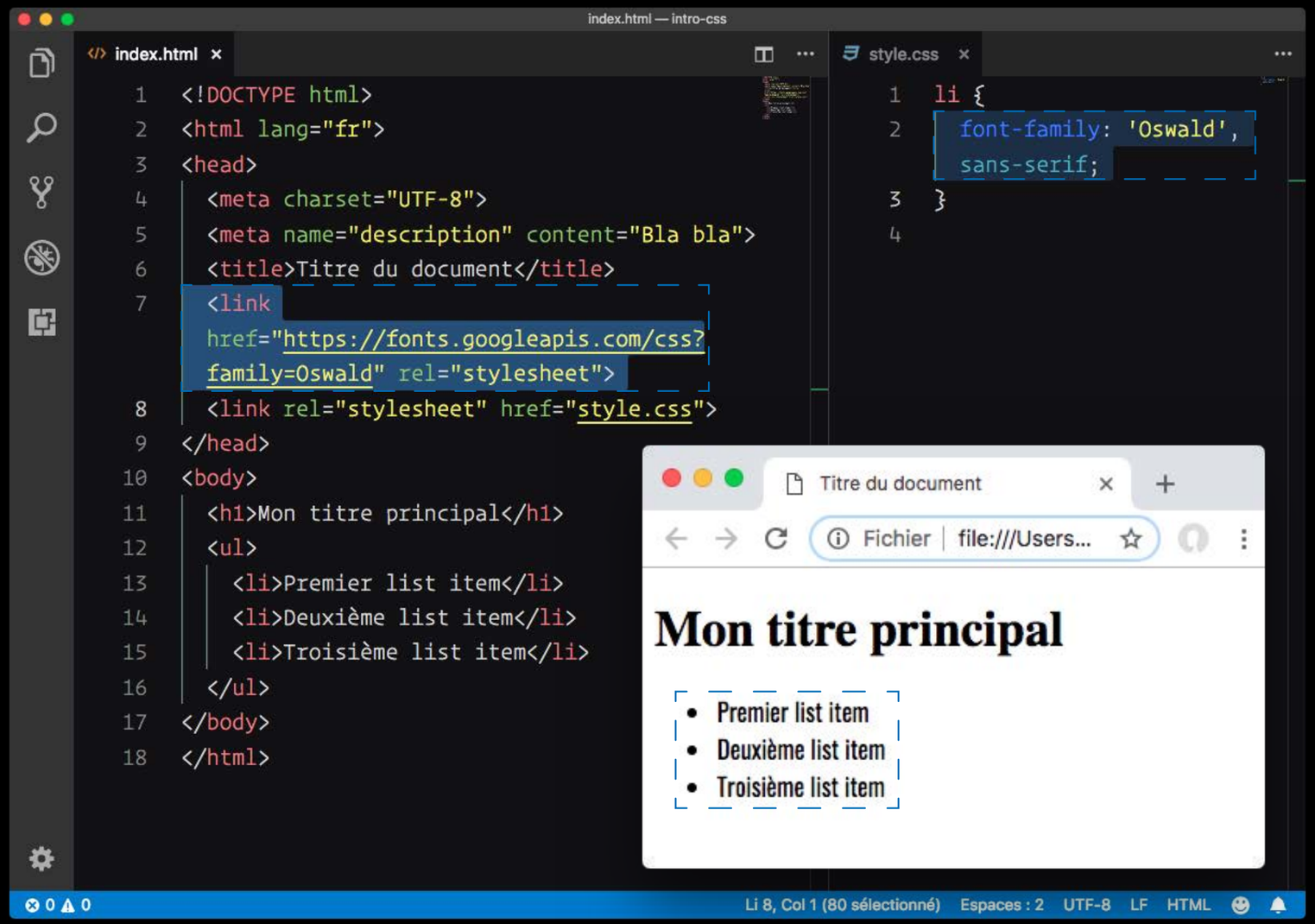
Utiliser une police hébergée par Google

Il nous faut maintenant **déplier cet onglet** qui nous affiche :

- la balise `<link>` que nous ajouterons **dans le `<head>` de notre page HTML**,
- **la règle CSS** que nous utiliserons **dans nos déclarations CSS**.



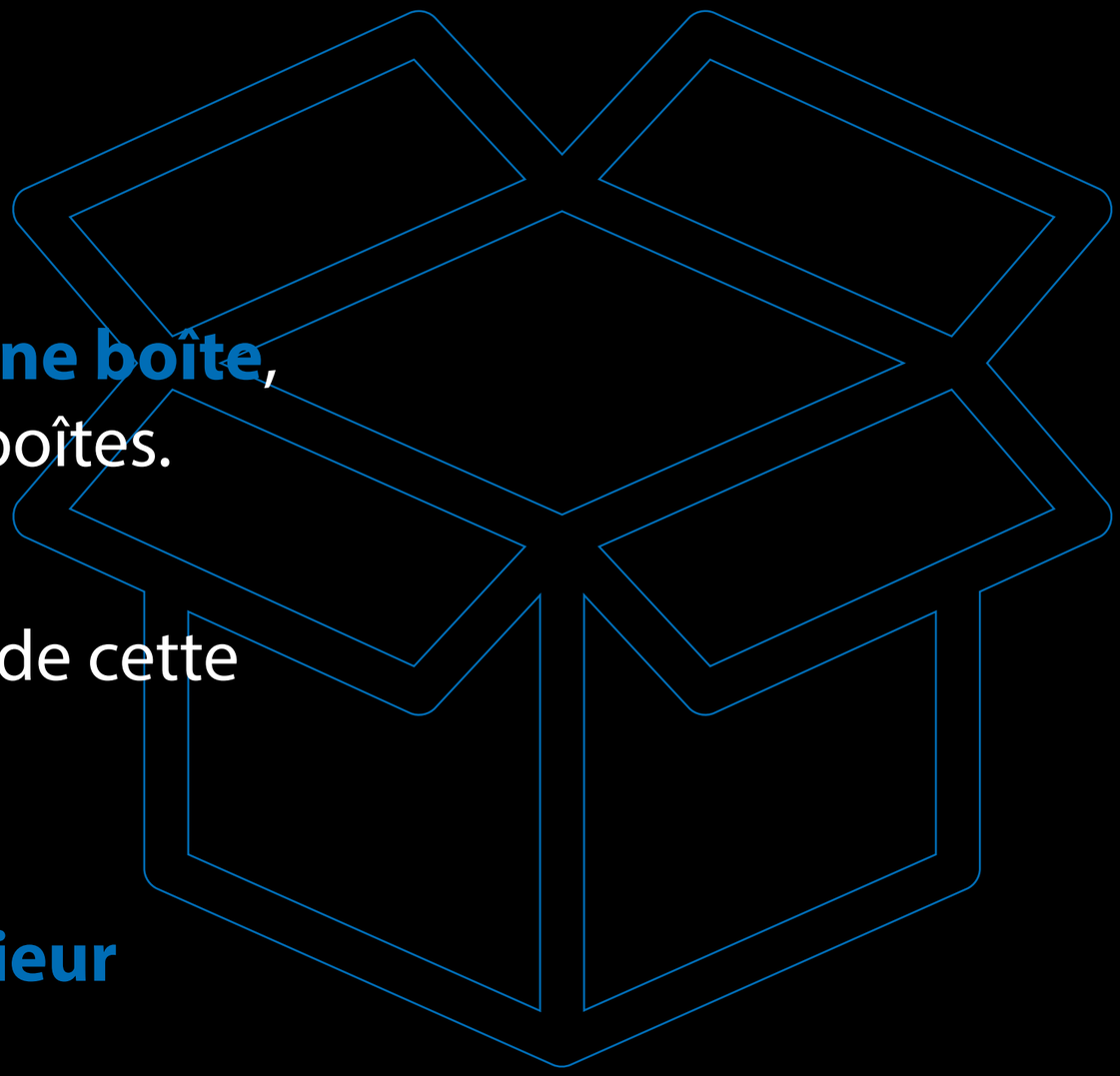
Utiliser une police hébergée par Google



Pour finir :

- nous collons dans le `<head>` de la page HTML la balise `<link>` fournie par Google Fonts,
- nous utilisons dans notre CSS la déclaration `font-family` fournie par Google Fonts.

Styliser une boîte

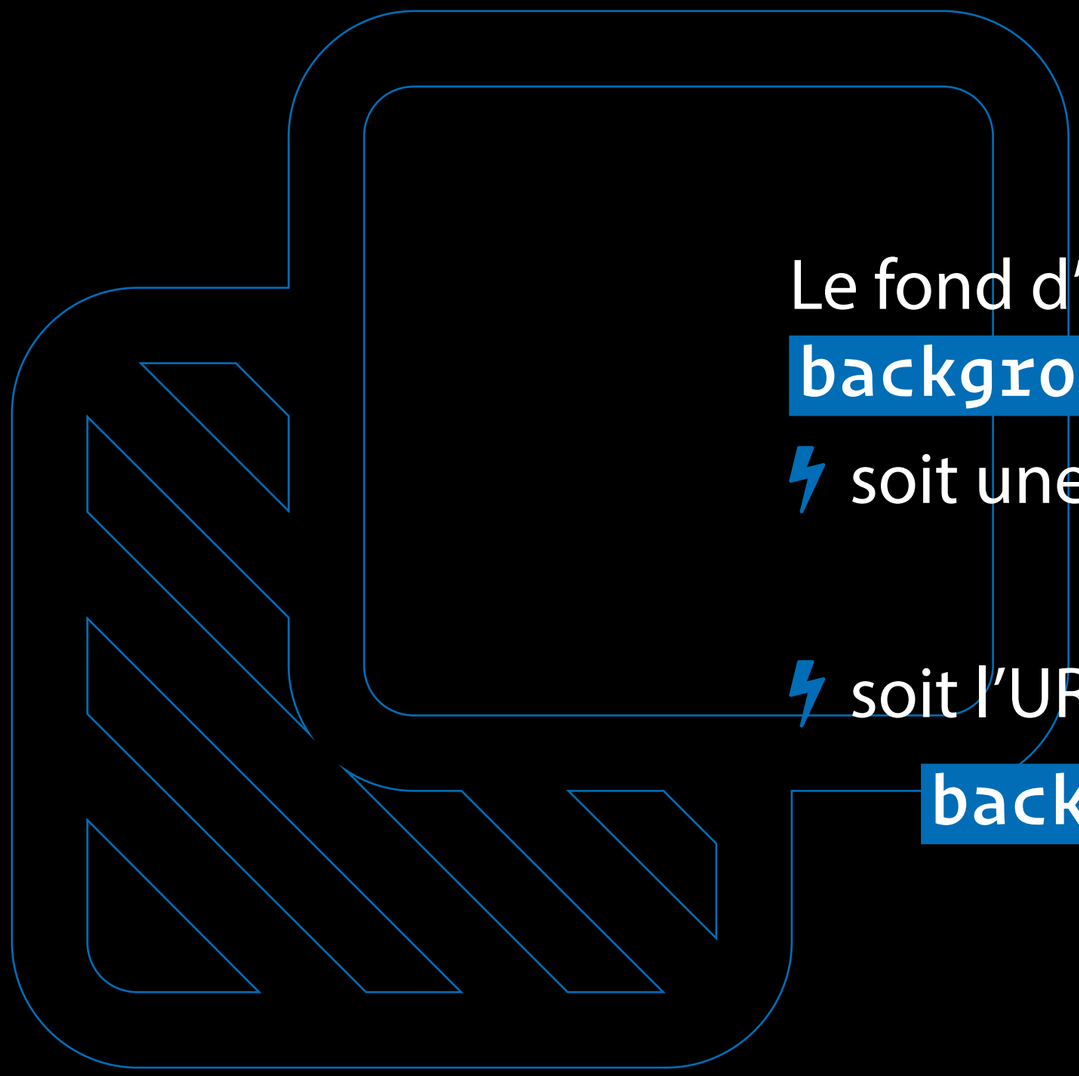


Il faut voir un élément HTML **comme une boîte**, qui peut elle-même contenir d'autres boîtes.

Les CSS nous permettent de définir :

- **les couleurs de fond** et **de contour** de cette boîte,
- **les dimensions** de cette boîte,
- **les espacements autour** et **à l'intérieur** de cette boîte.

Définir le fond de la boîte



Le fond d'une boîte est défini par sa propriété `background` à laquelle on donne pour valeur :

- soit une couleur, comme ceci :

```
background: #f00;
```

- soit l'URL d'une image, comme ceci :

```
background: url(mon-image.jpg);
```

Définir le fond de la boîte

Pour **afficher une image de fond correctement**, voici les attributs et leurs valeurs que nous utiliserons le plus fréquemment :

- `background: url(mon-image.jpg) center;`, où nous avons donné la valeur `center` en plus de l'URL pour que **le cadrage de l'image soit centré**,
- `background-size: cover;`, pour que **l'image couvre entièrement le fond** de la boîte, sans laisser d'espace vide, ni se répéter comme le motif d'une tapisserie.

Définir les contours de la boîte

Le contour de la boîtes se définissent avec la propriété `border` à laquelle on peut donner trois valeurs, sans ordre de préférence :

- l'épaisseur du contour (ex. : `2px`),
- le type de trait (par exemple `solid` pour un trait plein, `dotted` pour des pointillés, `dashed` pour des tiretés, etc.),
- une couleur (ex. : `#0f0`).

Voici l'exemple d'un trait plein vert de deux pixels d'épaisseur :

```
border: 2px solid #0f0;
```

Définir les contours de la boîte

La propriété `border` peut être suffixée de `-top`, `-right`, `-bottom` ou `-left` pour définir séparément les contours des quatres côtés d'une boîte.

Par exemple, nous définissons ici les quatre côtés d'une `<div>` :

```
div {
  border-top: 2px solid #0f0;
  border-right: 5px dashed #00f;
  border-bottom: 12px dotted rgba(255,0,0,0.8);
  border-left: 1px solid pink;
}
```

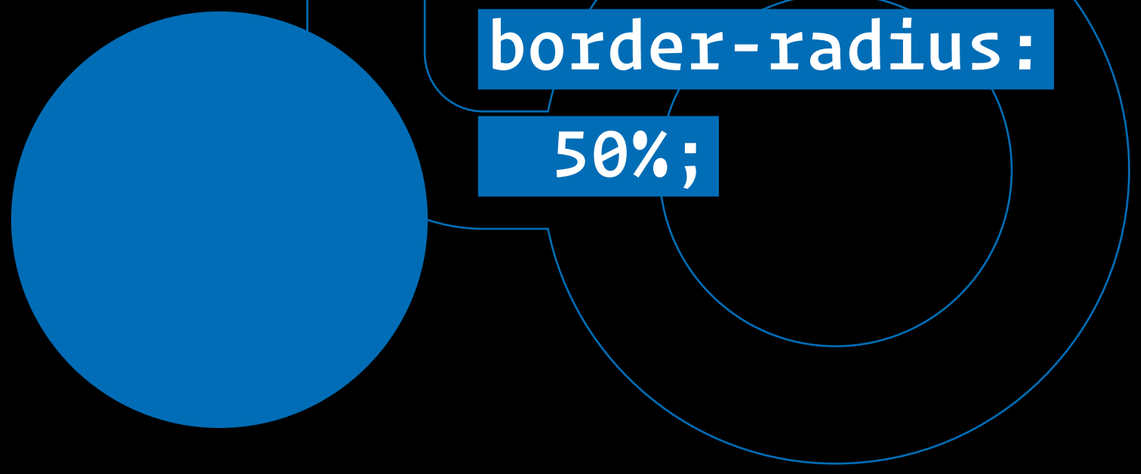
Définir l'arrondi des coins de la boîte

La propriété `border-radius` définit l'arrondi des coins d'une boîte. Nous lui donnerons pour valeur soit :

- **une dimension en pixels**, qui définit le rayon de l'arc de cercle qui constituera les coins,
- **un pourcentage** qui, à partir de 50 %, transformera une boîte rectangulaire en une ellipse.

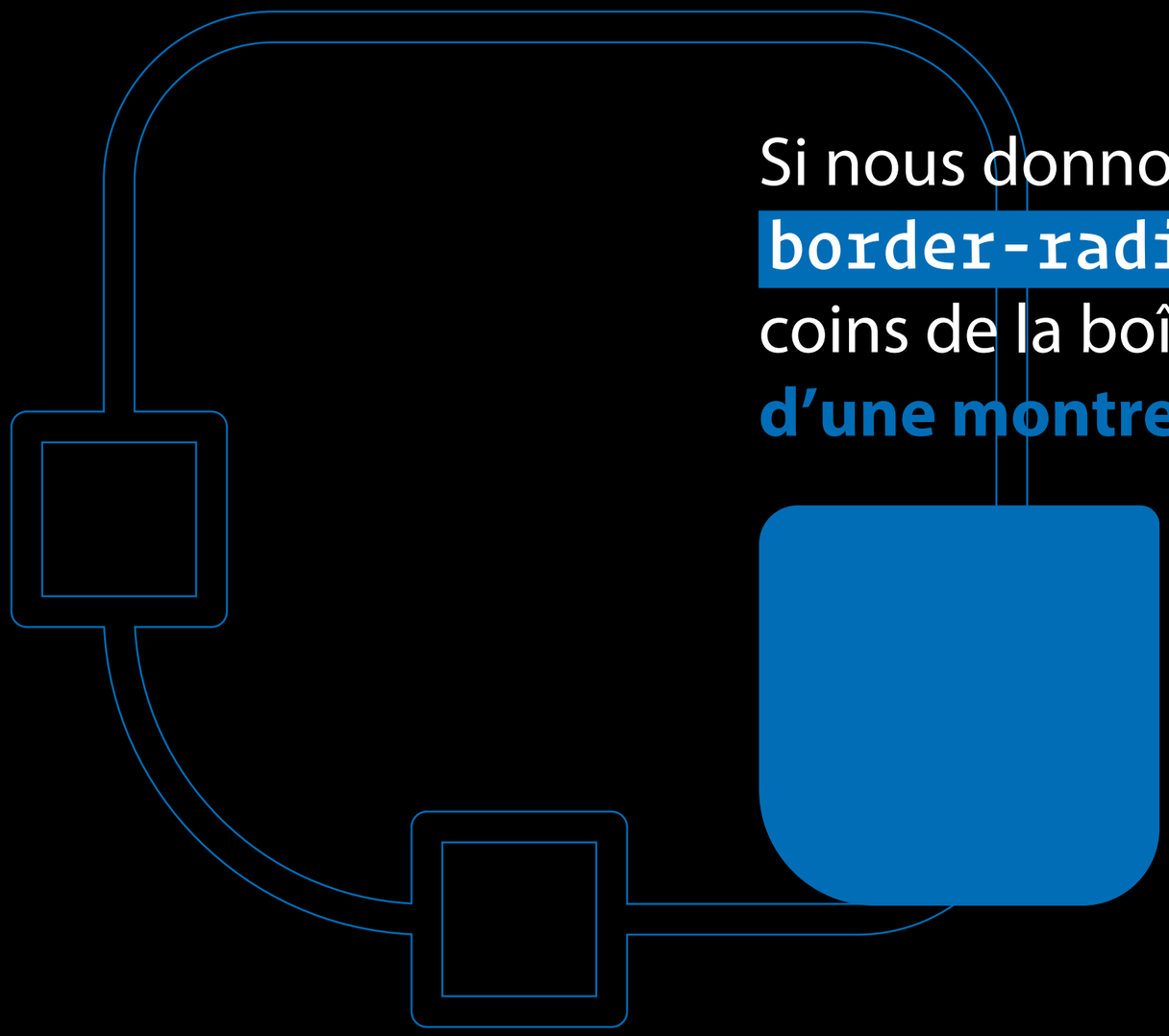


```
border-radius:
  20px;
```



```
border-radius:
  50%;
```

Définir l'arrondi des coins de la boîte



Si nous donnons **quatre valeurs** à la propriété `border-radius`, nous définissons les quatre coins de la boîte **dans le sens des aiguilles d'une montre** en partant du haut-gauche :

```
border-radius:
  20px 10px 40px 60px;
```

Définir l'arrondi des coins de la boîte

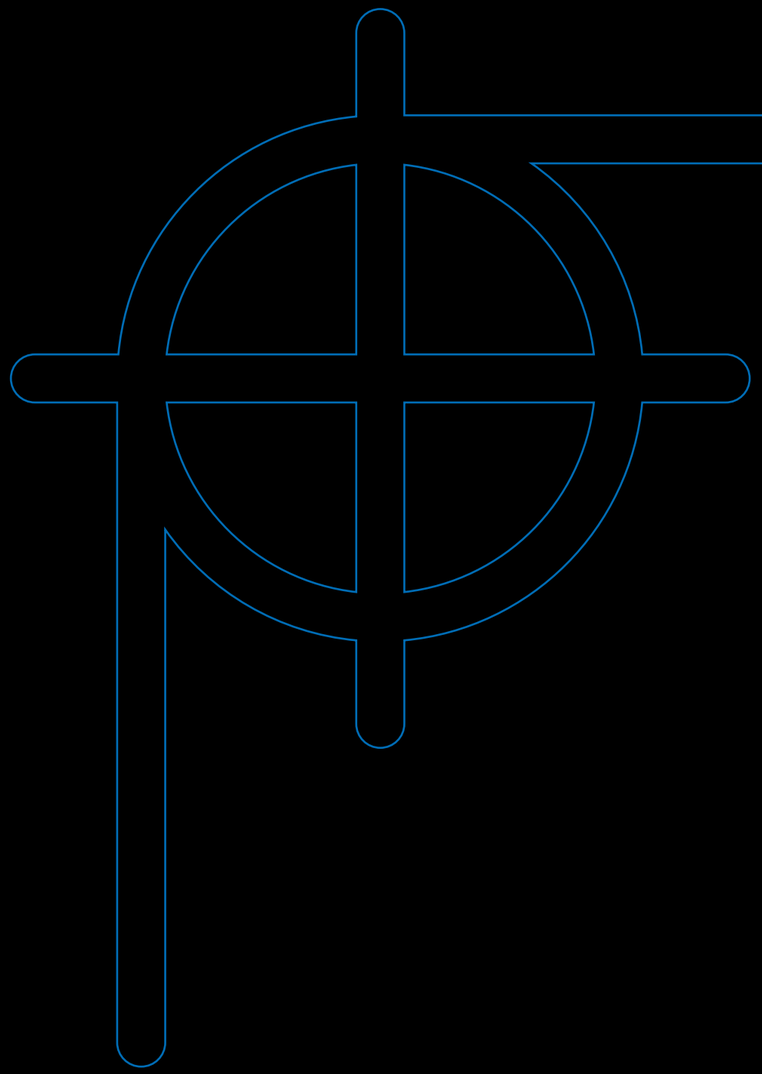
Si nous donnons **trois valeurs** à la propriété `border-radius` :

- **la première valeur** définit le coin **haut-gauche**,
- **la deuxième valeur** définit à la fois les coins **haut-droit** et **bas-gauche**,
- **la troisième valeur** définit le coin **bas-droit**.



```
border-radius:
  20px 10px 60px;
```

Définir l'arrondi des coins de la boîte



Enfin, si nous donnons **deux valeurs** à la propriété `border-radius` :

- **la première valeur** définit à la fois les coins **haut-gauche** et **bas-droit**,
- **la deuxième valeur** définit à la fois les coins **haut-droit** et **bas-gauche**



```
border-radius:
  40px 10px;
```

Définir l'arrondi des coins de la boîte

Nous pouvons aussi **définir les quatre coins indépendamment** avec ces propriétés :

- `border-top-left-radius`, pour définir l'arrondi du coin haut-gauche,
- `border-top-right-radius`, pour définir l'arrondi du coin haut-droite,
- `border-bottom-right-radius`, pour définir l'arrondi du coin bas-droit,
- `border-bottom-left-radius`, pour définir l'arrondi du coin bas-gauche.